SWINBURNE UNIVERSITY OF TECHNOLOGY

Artificial Intelligence Engineering
COS40007

Applied Project Task 1: System Blueprint & Project Planning

Dataset: Tetouan City Power Consumption Dataset

Semester 1, 2026

Jaspreet Singh - 105342118
James Scavone - 105872767
Thisal Weththasinghe - 104863391
Aidan McDonald - 105173059

Group Number: CL07_G03
Class Number: ACT627
Submission Date: 15 April 2026
Jira Board: Group Jira Board
GitHub Group Repository: GitHub Repository
Shared documentation Link: Group Shared Document

Table of Contents

Acknowledgement of Country *2*
1. Group Agreement and Project Plan *3*
1.1 Team Roles and Responsibilities 3
1.2 Timeline - Sprint Plan 3
1.3 Communication Plan 4
1.4 On-Track Behaviours 5
1.5 Off-Track Behaviours 5
1.6 Resolving Tensions 5
1.7 Team Expectations 5
2. System Blueprint Diagram *6*
2.1 Architecture Overview 6
2.2 Component Descriptions 7
2.3 Sprint Mapping 8
3. MLOps & DevOps Integration (High-Level Design) *9*
3.1 Dataset and Training Strategy 9
3.2 Retraining Strategy 9
3.3 Monitoring and Drift Detection 10
Performance Monitoring 10
Data Drift Detection 10
Concept Drift Detection 11
3.4 DevOps Design 11
Version Control Strategy 11
CI/CD Pipeline Concept 11
Code and Model Versioning 12
4. Project Management Setup (Agile Tools) *12*
4.1 User Stories 12
4.2 Backlog and Task Breakdown 13
4.3 Project Documentation 16
Shared Documentation Space 16
Project Description 16
System Overview 16
Key Design Decisions 17
5. Contribution Statement *18*
6. GenAI Declaration *18*

Jaspreet Singh Thisal Weththasinghe James Scavone Aidan McDonald

Acknowledgement of Country

We acknowledge the Wurundjeri Woi-wurrung and Bunurong Boon Wurrung peoples of the Eastern Kulin Nation as the Traditional Custodians of the land on which Swinburne University of Technology stands. We pay our respects to their Elders past, present, and emerging, and recognise their continuing connection to land, waters, and community. We also acknowledge all Aboriginal and Torres Strait Islander peoples across the country and their ongoing and important connection to Country.

1. Group Agreement and Project Plan

1.1 Team Roles and Responsibilities

The table below outlines each group member's assigned role within the project, along with their main responsibilities across the different stages. The roles were allocated based on each member's strengths and areas of interest, so overall the workload is distributed in a way that feels both fair and practical for the team.

Member & Student ID	Role	Key Responsibilities
Jaspreet Singh (105342118)	Scrum Master	Facilitates sprint planning and retrospectives, coordinates deadlines, prepares the Group Agreement and Contribution Statement, and manages final submission of deliverables.
Thisal Weththasinghe (104863391)	ML Engineer	Designs the system architecture diagram, builds and trains ML/forecasting models, and contributes to the MLOps & DevOps design sections.
James Scavone (105872767)	Data Engineer	Performs EDA and data preprocessing, develops the feature engineering pipeline, and sets up the Jira board and user stories.
Aidan McDonald (105173059)	DevOps Engineer	Creates and maintains the GitHub repository, compiles the final PDF submission, and manages the shared project documentation.

In this case, Jaspreet takes on the Scrum Master role across all sprints, which basically means he is responsible for keeping the group on track, organising sprint ceremonies, and making sure any blockers are identified and resolved as early as possible. This role also helps keep communication consistent across the team and ensures that deadlines are being monitored properly.

Overall, this role distribution also reduces unnecessary overlap between tasks. For example, separating data engineering, modelling, and DevOps responsibilities makes it easier to manage dependencies between different parts of the system, rather than having multiple people working on the same components at the same time.

1.2 Timeline - Sprint Plan

For this project, the work is divided into four sprints that align with the unit timeline and assessment schedule. Each sprint includes clearly defined goals, linked Jira tasks, and assigned members, which helps keep the workflow structured and easy to manage across the duration of the project.

Sprint	Goals & Deliverables	Jira Tasks	Owner(s)
Sprint 1 (01/04 - 15/04/2026)	Project setup, dataset selection, initial EDA, system blueprint draft, GitHub repo created, Jira board set up.	SCRUM-1: Project setup SCRUM-2: Dataset loading + EDA SCRUM-3: System design SCRUM-4: Documentation setup	All members - led by Jaspreet
Sprint 2 (15/04 - 06/05/2026)	Data preprocessing pipeline, feature engineering, baseline ML model, updated architecture diagram.	SCRUM-5: Data cleaning SCRUM-6: Feature engineering SCRUM-7: Baseline model SCRUM-8: Architecture update	Thisal, James

Sprint 3 (06/05 - 20/05/2026)	Model tuning, evaluation, MLOps design completed, documentation updates.	SCRUM-9: Model tuning SCRUM-10: Evaluation SCRUM-11: Documentation writing SCRUM-69: MLOps design	Thisal, Aidan
Sprint 4 (20/05 - 03/06/2026)	Final review, contribution statement, report compilation, submission.	SCRUM-12: Report compilation SCRUM-13: Final checks SCRUM-14: Submission SCRUM-70: GitHub finalisation	Jaspreet, Aidan

The sprint structure follows a logical progression - starting from initial setup and data understanding, moving through modelling and optimisation, and finishing with deployment and final submission. This basically ensures that each stage builds on the previous one rather than different components being developed in isolation from each other.

One thing that was also considered here is timing. Most sprint deadlines are set slightly before the official submission date where possible, which gives the group some buffer time for final reviews and fixing any unexpected issues before submitting. Sprint 1 in particular was quite important to get right, since a lot of the foundational decisions - dataset selection, repo setup, and documentation structure - all needed to be locked in early so the rest of the sprints could run smoothly.

It is also worth noting that Sprint 4 is intentionally lighter in terms of new development work. Most of the technical content is completed by the end of Sprint 3, and Sprint 4 focuses on making sure everything is properly documented, reviewed, and submitted. This was a deliberate choice to avoid rushing the final submission.

1.3 Communication Plan

To keep communication consistent and avoid confusion throughout the project, the group agreed on a straightforward communication plan at the start of Sprint 1. The idea was to keep things simple and practical, rather than overcomplicating it with too many different tools or channels.

- The primary communication channel is a WhatsApp group chat, which is used for quick updates, general discussions, reminders, and sharing any files that need quick feedback during the week.
- Weekly meetings are scheduled every Tuesday and are facilitated by the Scrum Master. These meetings are mainly used to review progress from the current sprint, discuss any blockers, and plan what needs to happen in the coming week.
- All shared work is stored in a shared Word document and the GitHub repository, where all members have full edit access. This makes it easy for everyone to stay up to date with what has been completed and what is still in progress.
- In terms of responsiveness, group members are expected to reply to messages within approximately 24 hours on weekdays. This also helps ensure that decisions don't get held up waiting for responses.
- Sprint planning and retrospective sessions are organised at the start and end of each sprint by the Scrum Master, to make sure the group stays aligned and any issues from the previous sprint are addressed before moving forward.

This setup was chosen mainly because it keeps day-to-day communication quick and flexible through WhatsApp, while still making sure more important discussions happen in a structured way during the weekly meetings. Having a dedicated meeting each week also helps the group catch any issues early rather than finding out too late in the sprint.

1.4 On-Track Behaviours

These are the kinds of behaviours the group agreed would lead to good outcomes throughout the project. Basically, if everyone is doing these things consistently, the project should run smoothly.

- Keeping communication consistent throughout each task - not just at the start and end of a sprint, but throughout the week as well.
- Making sure work is completed on time and to a reasonable standard before each sprint review, so nothing gets rushed at the last minute.
- Actively engaging with and responding to feedback from other group members, and being willing to share ideas and suggestions throughout the process rather than just working in isolation.
- Supporting other group members when they are struggling with a task, and being willing to help out or adjust the workload if something is taking longer than expected.
- Keeping the Jira board updated so that the current status of each task is always visible to the whole group, rather than having to ask for updates individually.

1.5 Off-Track Behaviours

These are the behaviours the group identified as potentially disruptive to the project. The idea here is not to create a list of punishments, but just to make it clear upfront what kinds of things could cause problems so everyone is on the same page from the start.

- Going unresponsive for extended periods without letting the group know - especially if it means tasks are not getting done or questions are not being answered.
- Not actively participating in assigned sprint tasks or being generally disengaged from the work, particularly without communicating about it.
- Creating unnecessary conflict or communicating with other group members in a way that is rude or disrespectful, which would make it harder for the team to work together effectively.
- Making significant changes to shared files or the GitHub repository without communicating it first, which could create confusion or overwrite someone else's work.

1.6 Resolving Tensions

Realistically, disagreements or tensions can come up in any group project, so the group agreed on a simple process for dealing with them if they do arise. The main goal here is to resolve things quickly and fairly without letting small issues grow into bigger problems.

1. The first step is to raise the concern directly with the person involved through the WhatsApp group chat or in a private message, in a respectful and constructive way. Setting aside time to work through the issue calmly is the priority here.
2. If the issue isn't resolved through that initial conversation, it gets raised openly during the next weekly Tuesday meeting, with the Scrum Master facilitating the discussion. Having the whole group present can sometimes help put things in perspective.
3. If the issue still can't be resolved between the group members themselves, the next step is to contact the unit tutor or convenor for guidance. This is a last resort, but it is there as an option if needed.
4. All decisions made through this process will be documented in the shared project folder so there is a clear record of what was discussed and agreed upon.

1.7 Team Expectations

At the start of the project, the group also set some general expectations for how everyone should approach their work and their interactions with each other. These are not meant to be formal rules, but more of a shared understanding of what is expected from each person.

- All team members will actively collaborate with each other throughout the project, providing constructive feedback and being open to hearing advice or suggestions from others.
- All work will be completed within the timeframe agreed on at the start of each sprint. If a member runs into difficulties, they are expected to communicate this early rather than waiting until the deadline.
- Accountability is important here. Each member is responsible for the tasks assigned to them, and the expectation is that work is delegated as evenly as possible across the team so no one person carries an unfair share of the load.

These expectations were deliberately kept straightforward, since overcomplicating a group agreement often means it doesn't get followed. The idea is just to make sure everyone is on the same page and knows what is expected of them from the start of the project.

2. System Blueprint Diagram

2.1 Architecture Overview

The diagram below shows the high-level architecture of the AI Energy Demand Forecasting System. To make it easier to follow, the system is organised into four logical tiers - a data pipeline tier, an ML pipeline tier, a deployment tier, and a monitoring and MLOps tier. Each tier handles a distinct phase of the AI Engineering lifecycle, and components are built and introduced progressively across sprints rather than all at once.

Data flows left-to-right through each tier, with cross-tier connections showing where outputs from one stage feed into the next. There is also a dashed red feedback loop running from the retraining trigger back to the ML model component, which represents the continuous improvement cycle that would be in place once the system is deployed. This helps ensure the diagram reflects not just how the system is built, but also how it would be maintained over time.

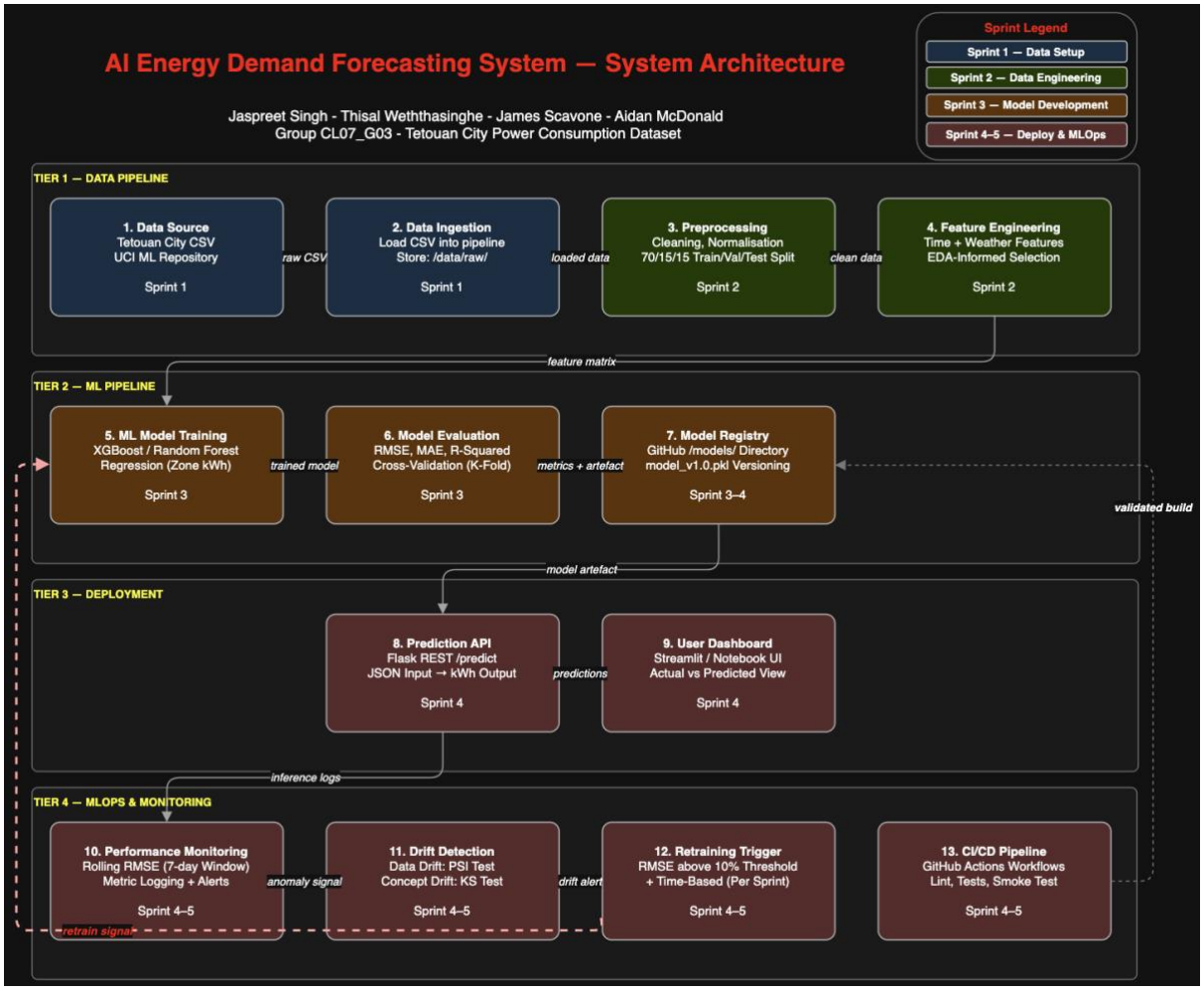

Figure 1: High-Level System Architecture - AI Energy Demand Forecasting System (Tetouan City Power Consumption Dataset, UCI ML Repository)

2.2 Component Descriptions

The system includes 13 components in total, spread across the four tiers. Each one is described below in order of data flow, along with a note on which sprint it is implemented in.

- Data Source (Sprint 1): The Tetouan City Power Consumption dataset from the UCI Machine Learning Repository. It contains around 52,417 hourly records covering all of 2017, with environmental features like temperature, humidity, and wind speed alongside zone-level power consumption as the target variable. It comes as a single structured CSV file.
- Data Ingestion (Sprint 1): A Python script that loads the raw CSV into the pipeline and saves it under /data/raw/ in the GitHub repository. One thing to note here is that the original file is never modified at this stage - anything that gets changed is saved separately downstream.
- Preprocessing (Sprint 2): This stage handles missing values using forward-fill to preserve temporal continuity, removes any duplicate rows, and also normalises numeric features using StandardScaler. The chronological 70/15/15 train/validation/test split is also applied here.
- Feature Engineering (Sprint 2): New features are extracted from the DateTime column - things like hour_of_day, day_of_week, month, and is_weekend - and combined with the existing weather features. These were selected based on EDA findings from Sprint 1 that showed strong correlations with consumption patterns. Processed data is saved to /data/processed/.

- ML Model Training (Sprint 3): A regression model (XGBoost or Random Forest) is trained on the engineered feature matrix, with zone-level kilowatt-hour consumption as the target. A simpler baseline model is also trained in Sprint 2 to give a benchmark to compare against.
- Model Evaluation (Sprint 3): The model is evaluated on the held-out test set using RMSE, MAE, and R-squared. Results are logged to a metrics file in the repo so performance can be tracked and compared across retraining cycles.
- Model Registry (Sprint 3-4): Trained model files are versioned and stored in the /models/ directory using a naming convention like model_v1.0.pkl. A model_registry.json file in the repo root also tracks each version's training date, dataset version, and evaluation metrics.
- Prediction API (Sprint 4): A lightweight Flask REST API that exposes a /predict endpoint. It accepts JSON-formatted input features and returns a predicted consumption value.
- User Dashboard (Sprint 4): A Streamlit-based dashboard that gives a visual overview of recent predictions, actual vs. predicted comparisons, and also model health indicators. This is mainly intended for the team and any stakeholders to keep an eye on how the model is performing.
- Performance Monitoring (Sprint 4-5): Rolling 7-day RMSE is tracked after each inference batch and logged. If it creeps above 10% of the original baseline RMSE, an alert fires and the retraining trigger is activated.
- Drift Detection (Sprint 4-5): PSI (Population Stability Index) is used weekly to detect data drift in input features. The KS (Kolmogorov-Smirnov) test is used to detect concept drift by comparing recent prediction distributions against the training-time distribution.
- Retraining Trigger (Sprint 4-5): Fires when either the RMSE threshold is exceeded or the scheduled sprint-milestone time arrives - whichever happens first. The retraining process then re-runs the full pipeline and only promotes the new model if it performs equal to or better than the current version.
- CI/CD Pipeline (Sprint 4-5): A GitHub Actions workflow that runs on every pull request to the dev and main branches. It runs linting with flake8, unit tests with pytest, and also a smoke test to confirm the model file is valid and produces sensible outputs.

2.3 Sprint Mapping

Each component is tied to the sprint where it is first introduced and implemented. This helps ensure the system evolves in a logical order rather than trying to build everything at once.

- Sprint 1: Data Source, Data Ingestion - the raw data is accessible, and the repository structure is in place.
- Sprint 2: Preprocessing, Feature Engineering - a clean, model-ready dataset is produced and stored.
- Sprint 3: ML Model Training, Model Evaluation, Model Registry - a trained, benchmarked, and versioned model is available.
- Sprint 4-5: Prediction API, User Dashboard, CI/CD Pipeline, Performance Monitoring, Drift Detection, Retraining Trigger - the full production-ready system is in place.

3. MLOps & DevOps Integration (High-Level Design)

3.1 Dataset and Training Strategy

The dataset the group has chosen is the Tetouan City Power Consumption dataset, sourced from the UCI Machine Learning Repository. It contains around 52,417 hourly records of electricity consumption across three distribution network zones in Tetouan, Morocco, all collected throughout 2017. Each record includes a datetime timestamp alongside six environmental features - temperature, humidity, wind speed, general diffuse flows, and diffuse flows - and the target variable is zone-level power consumption measured in kilowatts. The dataset comes as a single structured CSV file and is loaded directly into the pipeline from the /data/raw/ folder in the GitHub repository.

In terms of how the data gets split, a chronological split is used rather than a random one. This is a pretty important distinction for time-series data, because a random split would allow future observations to end up in the training set, which basically means the model would be learning from data it shouldn't have access to at prediction time. That kind of data leakage produces inflated performance metrics that don't reflect how the model would actually perform on unseen future data. So to avoid that, the data is split in order: 70% for training, 15% for validation, and 15% for testing. The training set covers roughly the first eight and a half months of 2017, the validation set covers the next two months and is used for hyperparameter tuning during Sprint 3, and the test set covers the final two months and is held out entirely until the final evaluation.

Since the Tetouan dataset is a static historical export rather than a live data stream, new data won't be ingested automatically during this project. Instead, the retraining strategy in Section 3.2 simulates a production retraining cycle by periodically re-evaluating the model against a rolling window drawn from the existing dataset. All raw files are stored in /data/raw/ and processed versions in /data/processed/, with versioned filenames like tetouan_processed_v1.csv to make sure results stay reproducible and the original data is never overwritten.

3.2 Retraining Strategy

Model retraining is an important consideration for any deployed ML system, mainly because the relationship between input features and the target variable can shift over time. With energy consumption data especially, things like seasonal changes, shifts in resident behaviour, or infrastructure changes could all cause the model to gradually become less accurate over time. Even though the Tetouan dataset is static, the retraining strategy here is designed to reflect how the system would be managed in a real production setting - so if the system were extended to use live data in the future, the same logic would apply.

The group decided to use two complementary trigger mechanisms rather than just one. Using a single trigger type has clear limitations: a performance-based trigger alone might miss gradual distributional shifts that haven't yet spiked the error metrics, while a time-based trigger alone might retrain the model unnecessarily when it's still performing well. Combining both gives a more complete safety net.

The primary trigger is performance-based. After each sprint review, the deployed model's predictions are evaluated against the most recent segment of the validation set. If the rolling RMSE exceeds the baseline RMSE (recorded at initial training) by more than 10%, a retraining cycle is started. The 10% threshold was chosen because it represents a meaningful enough drop in performance to warrant action, without being so sensitive that minor fluctuations constantly trigger unnecessary retraining. Since this is a regression task, RMSE is the main decision metric, though MAE and R-squared are also tracked as secondary indicators.

The secondary trigger is time-based. Regardless of what the RMSE looks like, a retraining cycle is scheduled at each sprint milestone - so roughly every three weeks. This mainly acts as a safety net to catch any slow, gradual drift that might not have shown up in the error metrics yet. It also lines up nicely with the sprint review ceremonies, which means there is always a fresh model available at the start of each new sprint.

When a retraining cycle fires, the process always follows the same steps: re-run the preprocessing and feature engineering pipeline on the full available dataset, retrain the model from scratch on the updated training partition, and evaluate it against the held-out test set. The new model only replaces the current one if it matches or improves on the existing performance. If it underperforms - which can happen when the new data is noisy or insufficient - the current model stays in place and the issue gets logged for discussion in the sprint retrospective. All model versions and their evaluation metrics are stored in /models/ with versioned filenames like model_v1.0.pkl and model_v1.1.pkl, making rollback straightforward if needed.

3.3 Monitoring and Drift Detection

Once the model is deployed, monitoring is basically what keeps the system reliable over time. You can't just train a model once and assume it will keep performing well indefinitely, especially with energy consumption data which tends to shift with seasons and usage patterns. So this section covers how the group plans to track model health and detect when something meaningful has changed.

Performance Monitoring

Performance monitoring of the ML model involves consistently tracking evaluation metrics throughout the model's lifecycle. For this project, since it's a regression task, the main metric being tracked is RMSE, which gives a good indication of how far off the model's predictions are from the actual values. MAE and R-squared are also tracked alongside it. These metrics are logged after each inference batch to a metrics file in the repository, so there's always a running record to compare against.

A rolling 7-day window is maintained specifically on the RMSE, and if that rolling value goes above 10% of the original baseline RMSE, an alert is triggered. This is also what feeds directly into the performance-based retraining trigger described in Section 3.2. Having this connected to the retraining logic means the response to degradation is somewhat automated rather than relying on someone manually checking logs. The metrics are also surfaced on the project dashboard so the team can visually monitor model health without having to dig through files.

Data Drift Detection

Data drift refers to when the statistical distribution of input features shifts relative to what the model was trained on. For this project, that's a real concern given the seasonal nature of the weather features - temperature and humidity values in summer are going to look very different from winter values, and if the model was trained predominantly on one season's data, its performance on another could degrade even if the model itself hasn't changed.

To detect data drift, the group will use the Population Stability Index (PSI), which compares feature distributions between a reference window (the training data) and a monitoring window (recent inference data). A PSI value above 0.25 is treated as a signal that the distribution has shifted significantly enough to warrant attention, and it feeds into the retraining trigger. PSI was chosen here because it's a fairly standard and interpretable approach for this kind of monitoring, and it works well for the continuous features that make up most of the input to this model.

Concept Drift Detection

Concept drift is a bit different from data drift. It's when the actual relationship between the features and the target variable changes, even if the feature distributions themselves still look fine. For example, if residents in Tetouan significantly change their electricity usage habits over time - maybe due to changes in building insulation, adoption of solar panels, or changes in industrial activity - the model's learned relationship between temperature and consumption might no longer hold.

To detect this, the group will apply the Kolmogorov-Smirnov (KS) test, which compares the distribution of model predictions over a recent window against the training-time distribution. If the test returns a p-value below 0.05, that's treated as statistically significant evidence of concept drift and also feeds into the retraining trigger. One thing to note here is that concept drift is generally harder to detect than data drift because there's no obvious change in the raw input data to point to - the inputs can look normal while the underlying relationship has shifted. That's why maintaining a sustained record of prediction error over time is also important, as a gradual upward trend in error is often a secondary indicator that concept drift may be occurring.

3.4 DevOps Design

Version Control Strategy

To ensure that all code changes are properly tracked and that backup versions always exist, version control is implemented through the group's shared GitHub Classroom repository, which has already been set up by Aidan. Every change pushed to the repository includes a descriptive commit message, which makes it easy to understand what was changed and why at any given point in the project's history. This also makes collaboration easier, since each group member can work on their own branch without impacting others.

A structured branching strategy is also in place to keep development separate from any finalised or production-ready code. The strategy works as follows:

- main: The protected core branch that only contains stable, reviewed code. No direct commits are made here - everything goes through a pull request.
- dev: The integration branch where all sprint work is merged first for testing before being promoted to main. This keeps unstable or in-progress code away from the production environment.
- feature/<task-id>: Short-lived branches for individual Jira tasks, named after the task ID for traceability (e.g., feature/SCRUM-5-data-cleaning). Once a task is complete, the branch is merged into dev via a pull request and then deleted.

All pull requests require at least one peer review from another group member before merging. Commit messages follow a consistent format: [SCRUM-ID] Short description - for example, [SCRUM-5] Add data cleaning pipeline. This keeps the git history clean and easy to trace back to specific tasks on the Jira board.

CI/CD Pipeline Concept

By having a CI/CD pipeline, the process of testing and validating code changes before they reach the repository is largely automated. This reduces the risk of human error and means only functional, tested code ends up in the production branch. A GitHub Actions workflow is planned for implementation in Sprint 4. The pipeline will be triggered automatically on every pull request to the dev and main branches and will run the following checks:

- Code linting and style checks using flake8, to catch formatting issues and obvious code quality problems before they get merged.

- Unit tests using pytest, covering the preprocessing, feature engineering, and model loading functions to make sure core pipeline components are working as expected.
- A model smoke test that loads the latest saved model file and runs a forward pass on a small synthetic input, just to confirm the model is valid and produces outputs within reasonable ranges.

No full production deployment automation is planned within the scope of this project, but having this CI pipeline in place means only tested and reviewed code ever reaches the main branch. This removes the manual deployment steps that would otherwise be needed and significantly reduces the chance of a broken model or faulty code reaching the end of the pipeline. The workflow file will be committed to /.github/workflows/ in the repository during Sprint 4.

Code and Model Versioning

Versioning is applied to both the code and the trained model files. This provides full traceability of the project's development history and makes it easy to identify when a particular feature was introduced or when a model change was made. Code versioning is handled through Git commits and tagged releases at the end of each sprint - for example, v1.0-sprint2 and v1.1-sprint3 - with significant milestones tagged to mark where important features or model optimisations occurred.

Trained model files are committed to the /models/ directory with a naming convention that includes the version number and training date, for example model_v1.2_20260501.pkl. There is also a model_registry.json file in the repository root that tracks each version's training date, the dataset version it was trained on, and its test set evaluation metrics. This way, the team can always tell which model is currently active and easily roll back to a previous version if a retrained model ends up performing worse than expected.

4. Project Management Setup (Agile Tools)

4.1 User Stories

The user stories below were defined to capture the key functional requirements of the system from the perspective of different stakeholders. Each story follows the standard Agile format - As a [user], I want [goal] so that [benefit] - which helps keep the focus on what each feature actually needs to deliver rather than just listing technical tasks.

ID	User Story	Priority	Sprint
US-01	As a data analyst, I want the raw data to be cleaned and processed so that I can have a high level of trust in the quality of the data I am inputting into the model.	High	Sprint 2
US-02	As a data engineer, I want to have a feature engineering pipeline that allows me to train the model on features that are based on important attributes.	High	Sprint 2
US-03	As an ML engineer, I want the baseline model that I set as a benchmark to be trained and evaluated so that I can reference it when comparing future improvements.	High	Sprint 2-3
US-04	As a project stakeholder, I want to have a system architecture diagram that demonstrates how each component interacts so that I can better visualise the system flow.	Medium	Sprint 1

US-05	As a developer, I want to have version control on the shared GitHub so that all changes made to the code are able to be tracked and rolled back if needed.	High	Sprint 1
US-06	As a team member, I want there to be a shared documentation space so that, if need be, I can access project decisions and track overall progress.	Medium	Sprint 1

These six stories cover the core requirements across data quality, feature engineering, model benchmarking, system visibility, version control, and documentation. In this case, the stories were deliberately kept broad enough to span multiple sprints, since breaking them into smaller subtasks is handled at the backlog level rather than at the story level. Each story is also assigned a priority level to make it clear which ones need to be addressed first during sprint planning.

4.2 Backlog and Task Breakdown

A Jira board was set up under the project name 'COS40007 Project' to manage the workflow and keep track of tasks across the development lifecycle. The board uses four columns - Backlog (not yet started), In Progress (currently being worked on), In Review (waiting on validation or feedback), and Done (completed) - which gives the whole team a clear view of where everything is at any given point in the sprint.

Each user story was broken down into smaller, actionable subtasks to ensure there's clarity around what actually needs to be done. For example, the data cleaning user story was split into specific steps like loading the dataset, handling missing values, removing duplicates, and exporting a cleaned version - rather than just leaving it as a single vague task. Tasks were then assigned to specific members based on their roles, distributed across the four sprints, and given a priority level (High, Medium, or Low) based on dependencies and overall impact.

The full task breakdown across all four sprints, directly reflecting the live Jira board, is shown in the table below:

Task ID	Sprint	Task Description	Assigned To	Status
SCRUM-1	Sprint 1	Project setup - initialise repo, set up environments, confirm group roles.	Jaspreet	Done
SCRUM-2	Sprint 1	Dataset loading + EDA - load Tetouan CSV, explore distributions, identify key features.	James	Done
SCRUM-3	Sprint 1	System Design - draft high-level architecture diagram in Draw.io.	Thisal	Done
SCRUM-4	Sprint 1	Documentation setup - create shared Word doc, establish project description and overview.	Aidan	Done
SCRUM-5	Sprint 2	Data cleaning tasks - handle missing values, remove duplicates, standardise formats.	Jaspreet	To Do
SCRUM-6	Sprint 2	Feature engineering tasks - create time-based and weather-derived features.	Aiden	To Do
SCRUM-7	Sprint 2	Baseline model tasks - train initial regression model, log RMSE/MAE/R^2.	James	To Do
SCRUM-8	Sprint 2	Architecture Update - revise blueprint to reflect implemented components.	Thisal	To Do
SCRUM-9	Sprint 3	Model tuning tasks - hyperparameter optimisation, cross-validation.	Thisal	To Do
SCRUM-10	Sprint 3	Evaluation tasks - final model benchmarking, comparison against baseline.	Aiden	To Do

SCRUM-11	Sprint 3	Documentation writing - update system overview and key design decisions.	Jaspreet	To Do
SCRUM-69	Sprint 3	MLOps design - Document retraining strategy, drift detection and monitoring plan.	James	To Do
SCRUM-12	Sprint 4	Report compilation - assemble all sections into final submission document.	Jaspreet	To Do
SCRUM-13	Sprint 4	Final checks - peer review, formatting, GenAI declaration, link verification.	Aiden	To Do
SCRUM-14	Sprint 4	Submission - export PDF, upload to Canvas, push final commit to GitHub.	James	To Do
SCRUM-70	Sprint 4	GitHub finalisation - Clean up repository, verify folder structure and push final commit.	Thisal	To Do

Overall, foundational tasks like project setup, data loading, and preprocessing were prioritised in the early sprints, while model optimisation, monitoring, and documentation tasks were pushed to later ones. This is mainly because the later tasks have dependencies on the earlier ones - you can't tune a model before it's been built, and you can't monitor something that hasn't been deployed yet.

The Jira board link and screenshots below show the current state of the backlog:
Jira Board: Group Jira Board

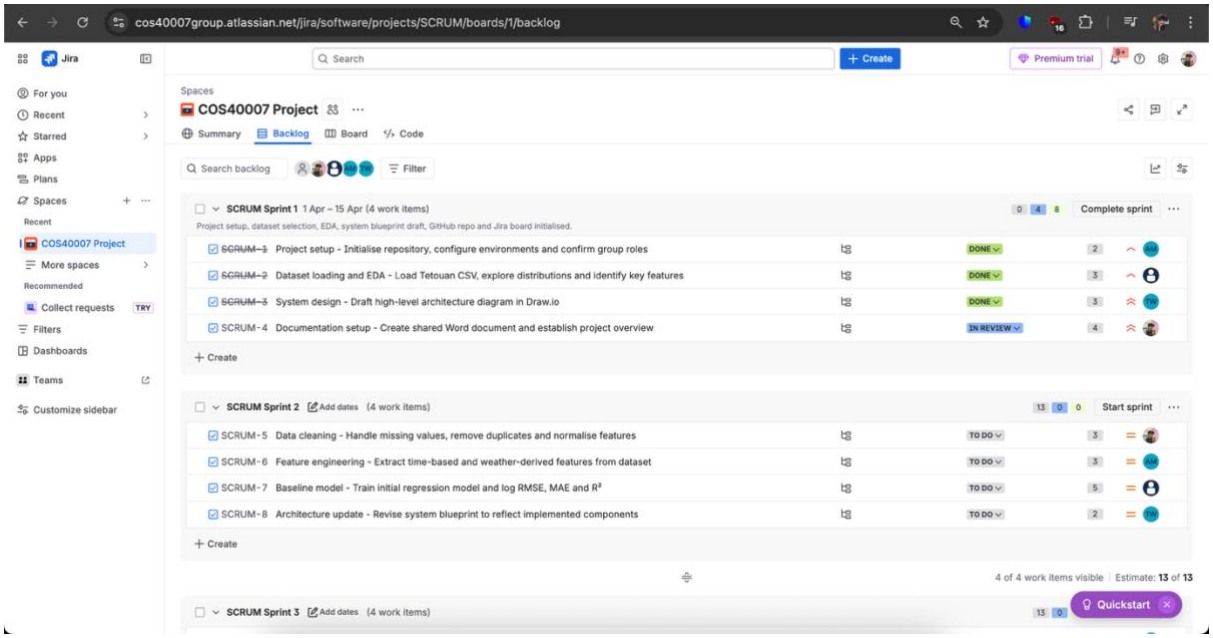

Figure 2: Jira Backlog - SCRUM Sprint 1 (SCRUM-1 to SCRUM-4) and SCRUM Sprint 2 (SCRUM-5 to SCRUM-8).

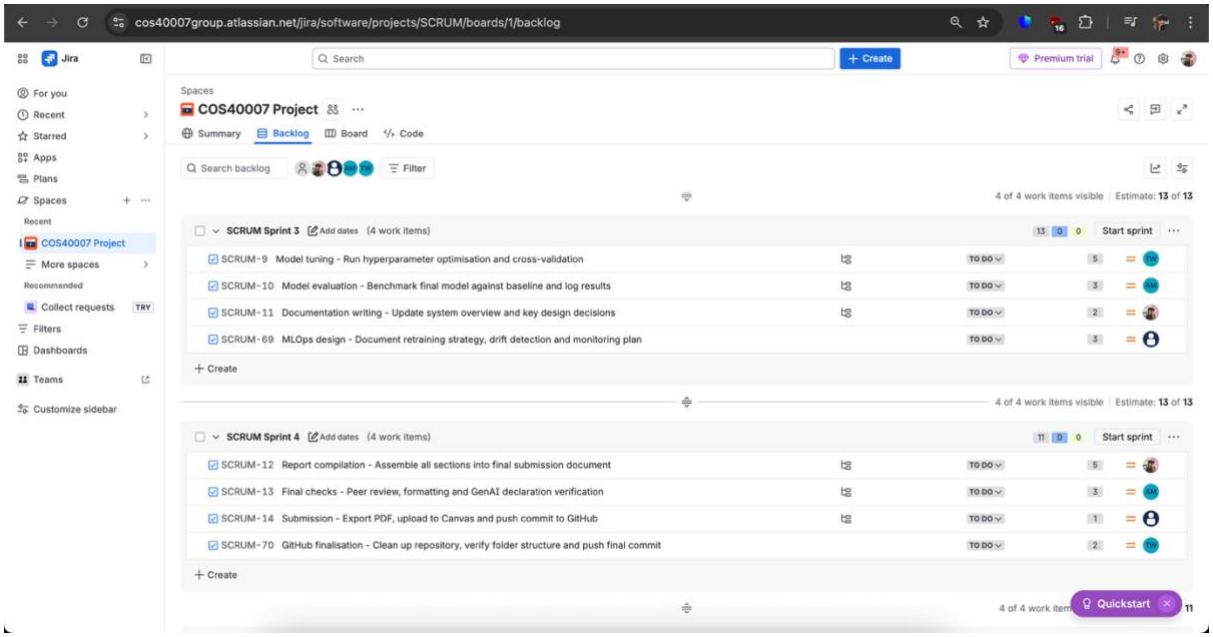

Figure 3: Jira Backlog - SCRUM Sprint 3 (SCRUM-9 to SCRUM-69 (Tried to rename 69 but wasn't allowed in Jira)) and SCRUM Sprint 4 (SCRUM-12 to SCRUM-70 (Tried to rename 70 aswell but the only possible way was to remake the whole Jira Board.).

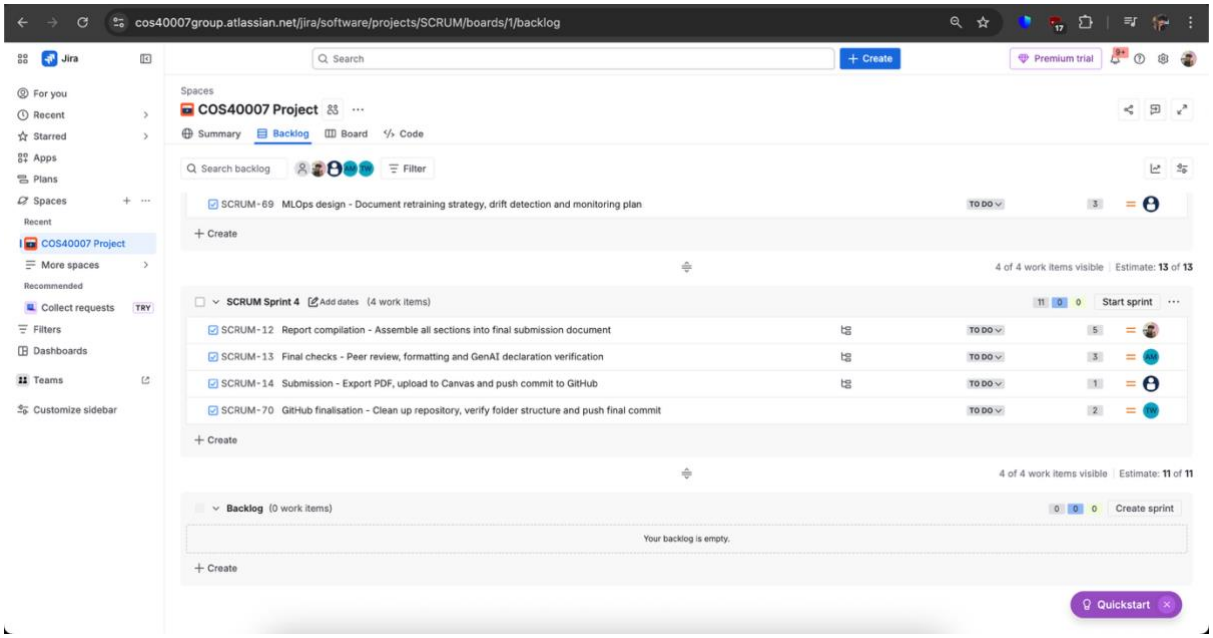

Figure 4: Jira Backlog - SCRUM Sprint 4 (SCRUM-12 to SCRUM-70) with empty backlog confirming all 14 tasks are fully allocated across sprints with no unplanned work remaining.

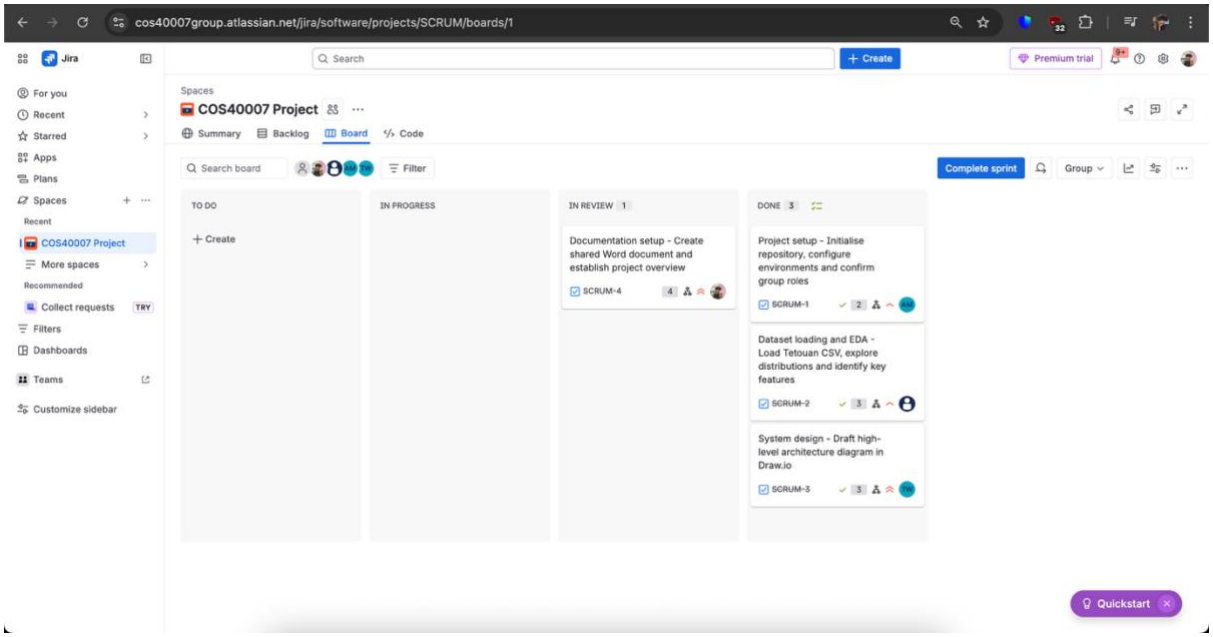

Figure 5: Jira Board View - SCRUM Sprint 1 active, showing task distribution across To Do, In Progress, In Review, and Done columns with assignees visible.

4.3 Project Documentation

Shared Documentation Space

A shared documentation space was set up using Microsoft Word to give all team members a central place to access, update, and track project information throughout the lifecycle. The idea here is to keep everything in one place so decisions don't get lost in chat messages and everyone is working from the same source of truth. All members have edit access, and the document is stored in the group's GitHub repository alongside the codebase so it stays version-controlled as well.

Project Description

This project is focused on building a machine learning system that can forecast electricity demand across three distribution zones in Tetouan, Morocco. The dataset being used is the Tetouan City Power Consumption dataset from the UCI Machine Learning Repository, which has around 52,000 hourly records covering all of 2017. Each record includes environmental features like temperature, humidity, and wind speed alongside the actual consumption values for each zone.

The main goal is to go through the full AI Engineering lifecycle - starting from loading and cleaning the raw data, through feature engineering and model training, all the way to evaluation and a conceptual MLOps strategy that covers monitoring, drift detection, and retraining. The whole project is structured across four Agile sprints, with each one building on the last so the system evolves incrementally rather than being developed all at once.

System Overview

The system is structured as a modular pipeline where each stage is kept separate from the others. This makes it easier to test, update, and maintain individual components without having to touch the whole codebase at once. The key stages are:

- Data Ingestion: Raw CSV data is loaded from the source and stored in /data/raw/ without any modifications to the original file.

- Data Preprocessing: Missing values are handled using forward-fill to preserve temporal ordering, duplicates are removed, features are normalised, and the data is split chronologically 70/15/15 to avoid data leakage.
- Feature Engineering: Time-based features (hour of day, day of week, month, is_weekend) and weather features are extracted and combined to form the final feature matrix, based on insights from the EDA in Sprint 1.
- Model Training: A regression model is trained on the feature matrix. A baseline model is also trained in Sprint 2 to serve as a benchmark for future comparisons.
- Model Evaluation: Performance is measured using RMSE, MAE, and R-squared on the held-out test set, with results logged to a metrics file in the repository.

A full system architecture diagram is included in Section 2 of the submission, which shows how all of these components connect and how data flows through the pipeline.

Key Design Decisions

A number of key decisions were made during the planning phase and are documented here so the group has a clear record of the reasoning behind the way the system was designed.

- Dataset Selection: The Tetouan City Power Consumption dataset was chosen because it combines time-series consumption data with environmental features, which makes it well-suited for regression modelling and feature engineering. It also doesn't have missing values in the original source, which simplifies the cleaning process.
- Chronological Data Split: A 70/15/15 chronological split was used instead of a random split to prevent data leakage in time-series data, where future observations could otherwise influence the training of past time periods.
- Feature Engineering Approach: Time-based features were derived from the DateTime column and combined with existing weather features, informed by EDA findings that showed strong correlations between these variables and consumption patterns.
- Model Selection: A baseline regression model is trained in Sprint 2 as a starting benchmark. The actual model selection happens in Sprint 3 after comparing performance across different algorithms.
- Modular Pipeline Design: Each stage of the pipeline is kept as a separate, independent component rather than being combined into one large script. This makes the system easier to test, debug, and reuse across different sprints.
- Version Control with GitHub: All code, notebooks, and model files are managed through the GitHub Classroom repository with a structured branching strategy to keep the main branch stable at all times.

All of these decisions were recorded in the shared documentation to maintain a clear project history and make it easier to revisit or adjust decisions in later sprints if needed.

Documentation Link: Group Shared Document

5. Contribution Statement

The table below documents each group member's contributions to Group Task 1. The workload was distributed as evenly as possible across the team, with each member taking ownership of the sections that aligned with their assigned role. All members have reviewed the final document and agree with the contribution breakdown.

Member & Student ID	**Contributions for Task 1**	**Share**
Jaspreet Singh 105342118	Group Agreement (Sections 1.1-1.7), Sprint Plan, MLOps Sections 3.1 and 3.2 (Dataset & Training Strategy, Retraining Strategy), Contribution Statement, final document submission via Canvas.	25%
Thisal Weththasinghe 104863391	System Blueprint Diagram (Section 2), architecture diagram in Draw.io, ML model selection and design, MLOps/DevOps overview (Sections 3.1-3.4 co-authoring).	25%
James Scavone 105872767	Data Engineering sections, Monitoring & Drift Detection (Section 3.3), DevOps Design (Section 3.4), Jira board setup, User Stories and Backlog (Sections 4.1-4.2).	25%
Aidan McDonald 105173059	GitHub repository creation and maintenance, Project Documentation (Section 4.3), PDF compilation, shared Word documentation space setup and maintenance.	25%

Overall, contributions were balanced across the group with each member contributing roughly 25% of the total work. Where tasks overlapped - for example, in the MLOps sections where both Jaspreet and Thisal contributed - this was coordinated during sprint meetings to avoid duplication and make sure the content was consistent throughout the document.

6. GenAI Declaration

In line with Swinburne University's Generative AI policy and the requirements for COS40007, the group is declaring the following use of AI tools in the preparation of this submission.

AI Tool Used	Section(s) Applied	How It Was Used	Modifications Made
ChatGPT (OpenAI)	Section 4.1 - User Stories	Used for idea generation to help the group brainstorm initial user story concepts based on the project requirements. It was not used to write any section of the document directly.	All user stories were reviewed by the group and rewritten in the group's own words to accurately reflect the actual project context and requirements.
ChatGPT (OpenAI)	All sections	Used for rubric clarification - the group asked ChatGPT to explain what each rubric criterion was looking for, and what elements a high-distinction response should include (e.g. what to write, why it matters, and how to structure the explanation). It was not used to generate any written content directly.	All content was written by group members. ChatGPT's guidance was used only to understand the depth and structure expected, not to produce the actual text.

The group confirms that all written content in this submission was produced by the group members themselves. ChatGPT was used as a brainstorming aid and rubric clarification tool during the planning phase, and no AI-generated text appears directly in this document.

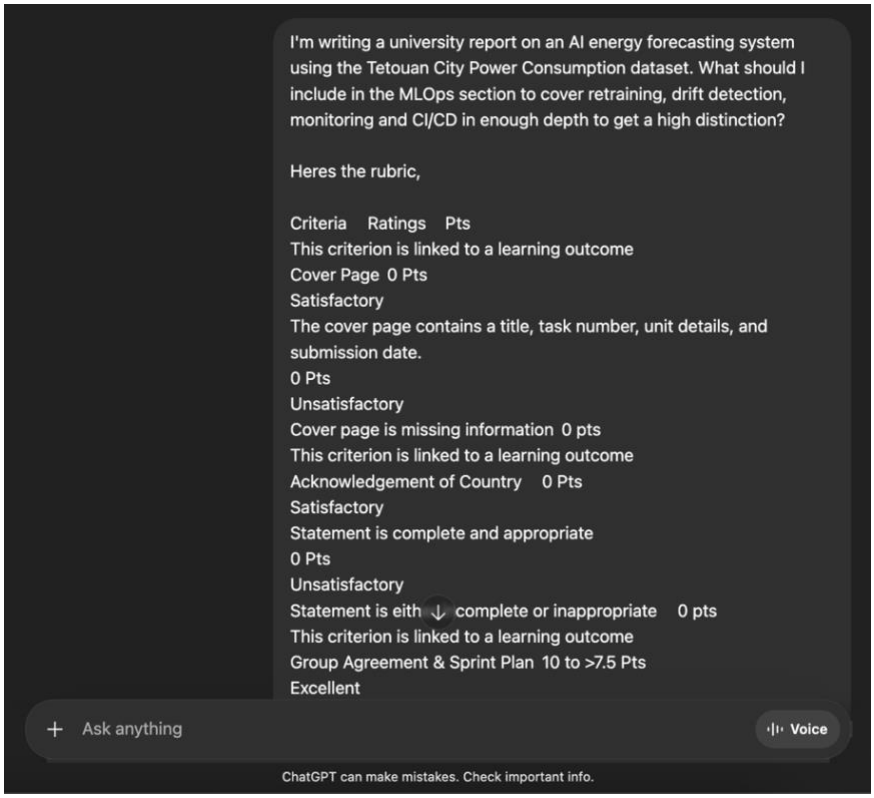

Image 1: ChatGPT prompt used to clarify rubric expectations for the MLOps and DevOps section.

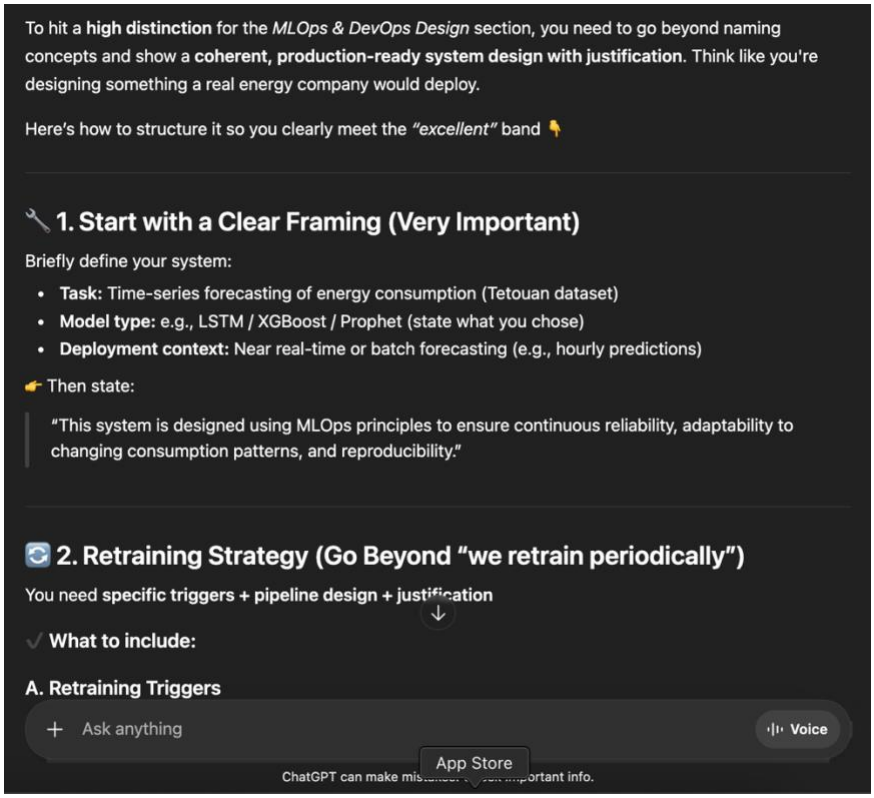

Image 2: ChatGPT response outlining what a high distinction MLOps section should include.

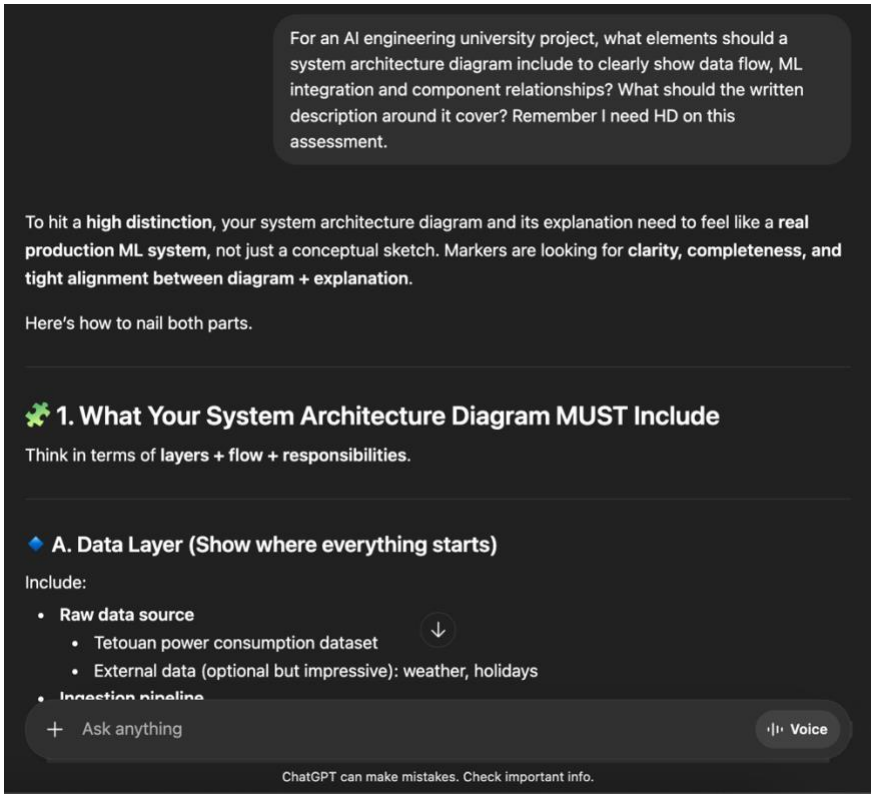

Image 3: ChatGPT prompt and response used to understand system architecture diagram requirements.

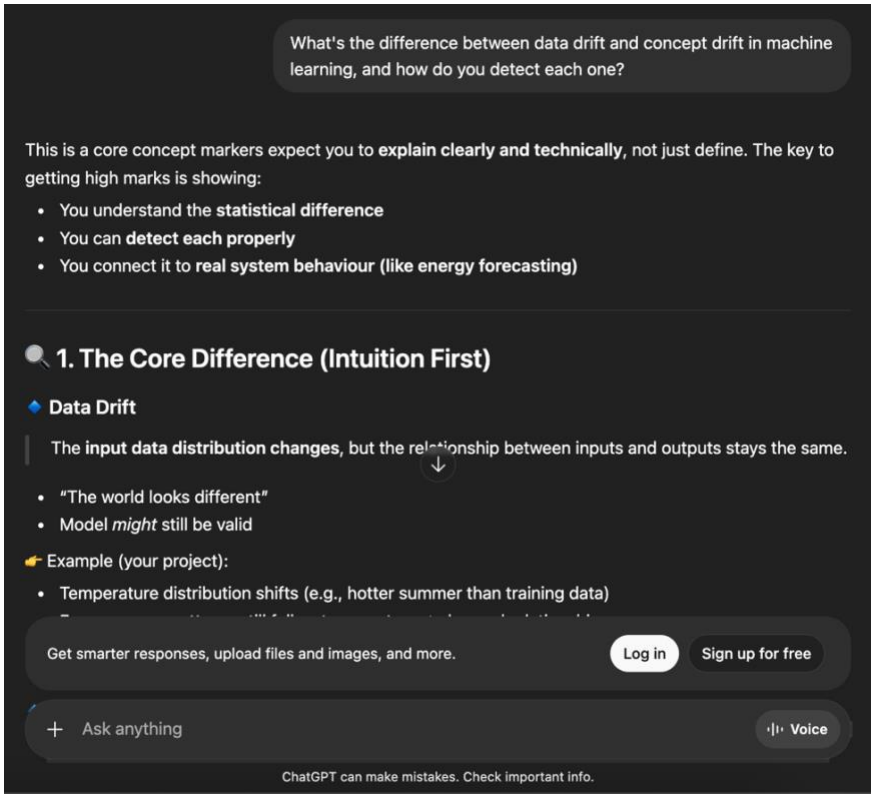

Image 4: ChatGPT response explaining data drift and concept drift for use in the monitoring section.